Q1 – Constructors and Constructor Overloading (6 Marks)

1. Design and implementation

A Book stores a title, author and number of copies. The default and title-only constructors delegate to the full constructor, so validation is implemented in one place. A negative copy count is rejected with std::invalid_argument.

```
#include <iostream>
#include <stdexcept>
#include <string>

class Book {
    std::string title;
    std::string author;
    int copies;

public:
    Book() : Book("", "Unknown", 0) {}
    explicit Book(const std::string& t) : Book(t, "Unknown", 0) {}
    Book(const std::string& t, int c) : Book(t, "Unknown", c) {}
    Book(const std::string& t, const std::string& a, int c)
        : title(t), author(a), copies(c) {
        if (copies < 0)
            throw std::invalid_argument("Number of copies cannot be negative");
    }

    void print() const {
        std::cout << "Title: " << title << ", Author: " << author
                  << ", Copies: " << copies << '\n';
    }
};

int main() {
    Book empty;
    Book donated("The Alchemist");
    Book limited("Clean Code", 5);
    Book complete("C++ Primer", "Lippman", 12);
    empty.print();
    donated.print();
    limited.print();
    complete.print();

    try {
        Book invalid("Invalid Book", "Unknown", -2);
    } catch (const std::invalid_argument& e) {
        std::cout << "Rejected: " << e.what() << '\n';
    }
}
```

2. Constructor concepts and analysis

- An implicit default constructor is generated only when the class has no user-declared constructor. Once any constructor is declared, the compiler does not automatically add a no-argument constructor.

- Member initializer lists directly initialise members. They are required for const members and references, avoid default-construct-then-assign work for class members, and make initialisation explicit.
- Members are initialised in the order in which they are declared in the class, not the order written in the initializer list. The initializer list should follow declaration order to avoid -Wreorder warnings and dependency bugs.
- Constructor delegation uses one constructor to call another constructor of the same class. It centralises validation and reduces duplicated logic.
- explicit prevents unintended implicit conversions through a one-argument constructor.

3. Object state after construction

Object	Title	Author	Copies
empty	(empty)	Unknown	0
donated	The Alchemist	Unknown	0
complete	C++ Primer	Lippman	12

4. Experimental investigation

- Run the program above and save the complete terminal output. The negative-count test should print a rejection message rather than construct an invalid object.
- To observe the missing-default-constructor error, remove Book() and try: Book shelf[3];. The compiler reports that it cannot find a matching zero-argument constructor for each array element. An array can instead be initialised explicitly, or a std::vector<Book> can be populated with emplace_back().
- To demonstrate -Wreorder, temporarily declare int copies before std::string title, but write an initializer list that initialises title before copies. Compile with -Wall -Wextra; the compiler warns that actual initialisation follows declaration order.
- Test copies = -1 and copies = 0. The former must throw; zero is a valid empty-stock count.

5. explicit demonstration

```
struct Distance {
    explicit Distance(int metres) {}
};
void show(Distance) {}
int main() {
    // show(5);       // Error: explicit constructor blocks conversion
    show(Distance(5)); // Correct direct construction
}
```

Without explicit, show(5) is allowed because the compiler implicitly converts 5 to Distance. With explicit, that implicit conversion is rejected; the caller must construct Distance explicitly.

6. Alternative: one constructor with default arguments

```
class Book {
    std::string title, author;
    int copies;
public:
```

```
    Book(std::string t = "", std::string a = "Unknown", int c = 0)
        : title(std::move(t)), author(std::move(a)), copies(c) {
        if (copies < 0) throw std::invalid_argument("Negative copies");
    }
};
```

Both designs create one object in O(1) time and use O(1) auxiliary space, excluding dynamically managed string storage. The single constructor with defaults centralises validation in one place, just like the delegated full constructor. Overloads communicate the supported creation situations more clearly and allow selected overloads to be marked explicit; default arguments provide a shorter interface but can make calls less expressive.

Q2 – Copy Constructors: Shallow and Deep Copies (7 Marks)

1. Why shallow copying fails

A compiler-generated copy of a raw pointer copies only its address. The original and copied Prescription then point to the same medicine array. A change through either object affects the shared array, and both destructors may attempt delete[] on the same address, causing a double-free error.

Shallow-copy memory view: p.medicines ──► [Paracetamol, ORS] ◄── q.medicines. Deep-copy view: p.medicines ──► [Paracetamol, ORS] and q.medicines ──► [Paracetamol, ORS] in a separate allocation.

2. Rule of Three and complete TODO solution

The Rule of Three says that a class managing a resource with a destructor will commonly need a copy constructor and copy-assignment operator as well. All three must agree on ownership and release rules.

```
#include <iostream>
#include <string>
#include <algorithm>

class Prescription {
    int n;
    std::string* medicines;

public:
    explicit Prescription(int count)
        : n(count), medicines(count > 0 ? new std::string[count] : nullptr) {}

    Prescription(const Prescription& other)
        : n(other.n),
          medicines(other.n > 0 ? new std::string[other.n] : nullptr) {
        for (int i = 0; i < n; ++i) medicines[i] = other.medicines[i];
        std::cout << "Copy constructor called\n";
    }

    Prescription& operator=(const Prescription& other) {
        if (this == &other) return *this;
        std::string* fresh =
            other.n > 0 ? new std::string[other.n] : nullptr;
        for (int i = 0; i < other.n; ++i) fresh[i] = other.medicines[i];
        delete[] medicines;
        medicines = fresh;
        n = other.n;
        return *this;
    }

    ~Prescription() { delete[] medicines; }

    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};

int main() {
    Prescription p(2);
    p.set(0, "Paracetamol");
```

```
    p.set(1, "ORS");
    Prescription q = p;
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p;
    p.set(0, "Ibuprofen");
    std::cout << p.get(0) << ' ' << p.get(1) << " | "
              << q.get(0) << ' ' << q.get(1) << " | "
              << r.get(0) << ' ' << r.get(1) << '\n';
}
```

Expected output (the additional copy-constructor message appears before the data line):

```
Copy constructor called
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
```

3. When the copy constructor is called

1. When an object is initialised from another object, for example Prescription q = p.
2. When an object is passed to a function by value (subject to permitted copy elision).
3. When an object is returned by value (copy elision or move construction may remove an actual copy).

Passing by const reference does not create a new Prescription object, so it does not invoke the copy constructor merely to pass the argument.

4. Self-assignment and sanitizer tests

The test p = p is safe because the assignment operator checks this == &other before allocating or deleting. The temporary fresh allocation is created before the old array is released, which also protects against allocation failure leaving the object without its original data.

- Correct deep-copy version: compile with g++ -std=c++17 -Wall -Wextra -fsanitize=address -fno-omit-frame-pointer -g prescription.cpp -o prescription and run it. Expected: no AddressSanitizer memory error.
- Shallow-copy experiment: remove the custom copy constructor but retain the destructor, then copy p. Both objects own the same pointer; AddressSanitizer is expected to report an attempting double-free or similar heap-use-after-free/double-free diagnostic when destruction occurs. Save the actual sanitizer report.
- Measure copies for 10, 1,000 and 100,000 medicine entries using std::chrono, repeat enough times to reduce noise, and record compiler, flags and elapsed time. Do not invent measured values.

5. Alternative: std::vector and complexity

```
#include <string>
#include <vector>

class Prescription {
    std::vector<std::string> medicines;
public:
    explicit Prescription(int count) : medicines(count) {}
    // Compiler-generated copy constructor and assignment are deep/value copies.
};
```

Copying n medicine strings is O(n) time and O(n) additional storage in both the raw-array deep-copy design and std::vector design. A shallow pointer copy is O(1) time and O(1) extra pointer storage, but it

is incorrect when both objects believe they own the same array. std::vector follows the Rule of Zero: its destructor, copy and assignment manage storage safely without custom resource-management functions.

The compiler-generated copy constructor can be relied on when all members already have correct value semantics (for example std::string, std::vector and numeric values). Write a custom copy constructor when the class directly owns a resource whose default member-wise copy would share ownership incorrectly.

Q3 – Dynamic Constructors and Destructors (7 Marks)

1. Dynamic seat-map implementation

The following class allocates a row-pointer array and then one char array for each row. A zero-row event is supported by storing a null row pointer.

```
#include <iostream>
#include <algorithm>
#include <stdexcept>

class Event {
    int rows, seats, booked;
    char** map;

public:
    Event(int r, int s) : rows(r), seats(s), booked(0), map(nullptr) {
        if (r < 0 || s < 0) throw std::invalid_argument("Negative dimensions");
        if (rows > 0 && seats > 0) {
            map = new char*[rows]{};
            try {
                for (int i = 0; i < rows; ++i) {
                    map[i] = new char[seats];
                    std::fill(map[i], map[i] + seats, 'O');
                }
            } catch (...) {
                for (int i = 0; i < rows; ++i) delete[] map[i];
                delete[] map;
                map = nullptr;
                throw;
            }
        }
        std::cout << "Created " << rows << "x" << seats << '\n';
    }

    bool book(int r, int s) {
        if (r < 0 || r >= rows || s < 0 || s >= seats || map[r][s] == 'X')
            return false;
        map[r][s] = 'X'; ++booked; return true;
    }
    bool cancel(int r, int s) {
        if (r < 0 || r >= rows || s < 0 || s >= seats || map[r][s] == 'O')
            return false;
        map[r][s] = 'O'; --booked; return true;
    }
    ~Event() {
        for (int i = 0; i < rows; ++i) delete[] map[i];
        delete[] map;
        std::cout << "Closed event; booked seats at closing: " << booked << '\n';
    }
};

int main() {
    Event local(2, 3);
    local.book(0, 1);
    {
        Event blockEvent(1, 2);
```

```
        blockEvent.book(0, 0);
    } // blockEvent is destroyed here
    Event* heapEvent = new Event(2, 2);
    heapEvent->book(1, 1);
    delete heapEvent; // required to invoke destructor and release memory
    Event empty(0, 5);
}
```

2. Object lifetime and memory layout

- Automatic local objects are destroyed in reverse order of construction when their scope ends. A block-scope object is destroyed at the closing brace of that block.
- A dynamically allocated object created with new remains alive until delete is called. Forgetting delete leaks the object and its allocations, and its destructor is not called.
- An array allocated with new[] must be released with delete[]. Using scalar delete on an array is undefined behaviour; using delete[] on a scalar allocation is also undefined behaviour.
- A destructor has no parameters and no return type. A class has one destructor, so it cannot be overloaded. It is invoked automatically as object lifetime ends or explicitly through delete for a dynamically allocated object.

Memory layout for a 3-row, 4-seat map:

```
Event.map ──► row-pointer array [ptr0 | ptr1 | ptr2]
                ptr0 ──► ['O','O','O','O']
                ptr1 ──► ['O','O','O','O']
                ptr2 ──► ['O','O','O','O']
```

3. Logger program: predicted output

Write the prediction before compiling. The exact expected sequence is:

```
+A +B -B copyAc use -Ac +P -P end -A
```

Explanation: a is constructed first. b is constructed and destroyed when its inner block ends. Passing a by value to use creates a copy named Ac; the function prints use, then the parameter copy is destroyed at the end of the call. The heap object P is constructed by new and destroyed by delete. The final local object a is destroyed at the end of main.

4. RAII and experimental investigation

RAII (Resource Acquisition Is Initialization) ties resource ownership to an object's lifetime. The constructor acquires the seat-map memory and the destructor releases it automatically, including during normal scope exit. This reduces forgotten cleanup and makes ownership easier to reason about.

- Run once with the delete heapEvent line removed. AddressSanitizer should report a memory leak if leak detection is enabled. Restore delete and verify that the leak report disappears.
- Test Event(0, 5), Event(10, 20), Event(100, 200) and Event(1000, 2000). A char seat requires approximately one byte, so the seat payloads are 200 B, 20,000 B and 2,000,000 B respectively, plus row pointers and allocator metadata. These are theoretical payload sizes, not measured resident-memory figures.
- Record constructor/destructor messages to verify lifetime order. Save actual sanitizer output and measured memory evidence.

5. Alternative memory layouts and complexity

Representation	Allocations	Access	Extra row-pointer storage
char** with separately allocated rows	rows + 1	O(1)	rows pointers
One contiguous char block	1	O(1)	None
vector<vector<char>>	Typically rows + 1 internal allocations	O(1)	Vector objects/metadata per row

A contiguous representation uses index r * seats + s. It has one allocation, better locality and no separate row-pointer array. All three approaches provide O(1) indexed access. The row-pointer design uses O(rows) extra pointer storage and more allocations.

Q4 – Overloading Arithmetic and Stream Operators (7 Marks)

1. Money class and completed TODOs

```
#include <iomanip>
#include <iostream>

class Money {
    long long paise;

public:
    Money(long long rupees = 0, int p = 0)
        : paise(rupees * 100 + p) {}

    Money operator+(const Money& other) const {
        Money result;
        result.paise = paise + other.paise;
        return result;
    }
    Money operator-(const Money& other) const {
        Money result;
        result.paise = paise - other.paise;
        return result;
    }
    Money& operator+=(const Money& other) {
        paise += other.paise;
        return *this;
    }
    bool operator==(const Money& other) const {
        return paise == other.paise;
    }
    bool operator<(const Money& other) const {
        return paise < other.paise;
    }
    friend std::ostream& operator<<(std::ostream& out, const Money& m) {
        long long whole = m.paise / 100;
        long long fraction = m.paise % 100;
        if (fraction < 0) fraction = -fraction;
        if (m.paise < 0 && whole == 0) out << "-";
        out << "Rs " << (whole < 0 ? -whole : whole) << '.'
            << std::setw(2) << std::setfill('0') << fraction
            << std::setfill(' ');
        return out;
    }
};

int main() {
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    std::cout << c << '\n' << std::boolalpha << (c == Money(200)) << '\n';
    Money x(0, 75), y(0, 50);
    std::cout << x + y << '\n';
}
```

Expected output:

```
Rs 200.00
true
```

```
Rs 1.25
```

2. Operator design

- + returns a new value and leaves its operands unchanged. += modifies the left operand and returns Money& (*this), allowing chaining without an unnecessary result copy.
- Arithmetic and comparison operators can be members because they naturally use the left operand's private paise field. Stream insertion must be a non-member because the left operand is std::ostream, not Money; it is declared friend to access paise.
- The expression a + b is interpreted as a.operator+(b) for a member overload, or operator+(a, b) for a non-member overload.
- An extraction operator >> can also be a friend non-member. A robust input format should define whether input is rupees plus paise or a decimal amount, then validate it.

3. Money engineering decisions and tests

Adding and subtracting money, equality and ordering are meaningful. Multiplying one Money amount by another Money amount is generally not meaningful as a monetary operation; multiplication by a scalar quantity (for example a count or tax rate) is more appropriate. Negative amounts may be permitted for refunds/credits, or rejected by a business rule; this implementation stores signed paise and supports negative results.

A double-based representation can introduce binary floating-point rounding. Integer paise gives exact cent/paise arithmetic for whole minor units. For one million additions, both implementations perform O(1) work per addition and O(1) total storage; actual runtime must be benchmarked on the same compiler and machine. A loop that repeatedly adds 0.01 to a double may demonstrate accumulated representation error.

4. Alternative: non-member operators or named functions

```
Money operator+(const Money& a, const Money& b) {
    return Money(/* combined amount */);
}
// Named-function alternative:
// Money add(const Money& a, const Money& b);
// bool equals(const Money& a, const Money& b);
```

Member and non-member arithmetic overloads both take O(1) time and O(1) auxiliary space. A non-member overload can support conversions on either operand when designed appropriately; for example, a non-member operator+(Money, Money) with a non-explicit converting constructor can allow both 100 + m and m + 100. A member operator+ is called on the left operand and cannot make 100.operator+(m) valid.

Q5 – Overloading Increment, Comparison and Subscript Operators (6 Marks)

1. Time and Playlist implementation

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

class Time {
    int seconds;
public:
    explicit Time(int s = 0) : seconds(s) {
        if (s < 0) throw std::invalid_argument("Negative time");
    }
    Time& operator++() { ++seconds; return *this; } // prefix
    Time operator++(int) {
        Time old(*this); ++seconds; return old;     // postfix
    }
    bool operator==(const Time& t) const { return seconds == t.seconds; }
    bool operator<(const Time& t) const { return seconds < t.seconds; }
    int totalSeconds() const { return seconds; }
    void print() const {
        int h = seconds / 3600, m = (seconds / 60) % 60, s = seconds % 60;
        std::cout << h << ':' << (m < 10 ? "0" : "") << m
                  << ':' << (s < 10 ? "0" : "") << s;
    }
};

class Playlist {
    std::vector<std::string> tracks;
public:
    explicit Playlist(std::vector<std::string> t) : tracks(std::move(t)) {}
    std::string& operator[](int i) {
        if (i < 0 || i >= static_cast<int>(tracks.size()))
            throw std::out_of_range("Playlist index out of range");
        return tracks[i];
    }
    const std::string& operator[](int i) const {
        if (i < 0 || i >= static_cast<int>(tracks.size()))
            throw std::out_of_range("Playlist index out of range");
        return tracks[i];
    }
    int size() const { return static_cast<int>(tracks.size()); }
};

int main() {
    Time t(5);
    Time a = t++;
    Time b = ++t;
    std::cout << a.totalSeconds() << ' ' << b.totalSeconds()
              << ' ' << t.totalSeconds() << '\n';

    Playlist p({"Song A", "Song B"});
    std::cout << p[0] << '\n';
    try { std::cout << p[p.size()] << '\n'; }
    catch (const std::out_of_range& e) { std::cout << e.what() << '\n'; }
```

```
    Time rollover(3599);
    ++rollover;
    rollover.print(); std::cout << '\n';
}
```

2. Prefix and postfix rules

- Prefix is declared operator++() and returns a reference to the already incremented object.
- Postfix is declared operator++(int). The dummy int distinguishes it from prefix. Postfix saves and returns the old value, so it normally makes a copy.
- Playlist needs both const and non-const operator[] overloads. A non-const playlist returns std::string& so callers can modify a track; a const playlist returns const std::string& to prevent modification.
- An invalid index should not silently access memory. This implementation throws std::out_of_range for negative indices and indices greater than or equal to size.

3. Predict the given Counter output

Given c starts at 5. c++ returns the old value 5 and changes c to 6. ++c changes c to 7 and returns the updated value 7. Therefore:

```
5 7 7
```

Postfix makes a copy of the old object; prefix does not need to make that old-value copy.

4. Rollover and performance experiments

The Time class above stores total seconds, so 59:59 is 3,599 seconds and one increment produces 3,600 seconds, displayed as 1:00:00. Test playlist[-1] and playlist[size] and verify that both throw std::out_of_range.

Benchmark 10,000,000 prefix and postfix increments separately at -O0 and -O2. Use identical loops and ensure the final value is consumed (for example, print it or store it in a volatile sink) so the optimizer cannot remove the work. Record actual times; optimized builds may make the difference very small.

5. Alternative access designs and complexity

Access method	Time per access	Behaviour on invalid index
Checked operator[] (this design)	O(1), one bounds comparison	Throws out_of_range
vector::at(i)	O(1), bounds check	Throws out_of_range
Unchecked operator[]	O(1), no explicit check	Out-of-range access is undefined behaviour

All methods have O(1) access complexity. The check adds a constant-time comparison; measure 10,000,000 accesses with checked and unchecked versions under the same optimization level to estimate its practical cost. Many programmers prefer ++i for class types because prefix avoids constructing and returning the old temporary value.

Q6 – The Rules of Operator Overloading (7 Marks)

1. Text class using dynamic character storage

```
#include <cstring>
#include <iostream>
#include <utility>

class Text {
    char* data;
public:
    Text(const char* s = "") {
        std::size_t n = std::strlen(s);
        data = new char[n + 1];
        std::memcpy(data, s, n + 1);
    }
    Text(const Text& other) {
        std::size_t n = std::strlen(other.data);
        data = new char[n + 1];
        std::memcpy(data, other.data, n + 1);
    }
    Text& operator=(const Text& other) {
        if (this == &other) return *this;
        std::size_t n = std::strlen(other.data);
        char* fresh = new char[n + 1];
        std::memcpy(fresh, other.data, n + 1);
        delete[] data;
        data = fresh;
        return *this;
    }
    ~Text() { delete[] data; }

    Text operator+(const Text& rhs) const {
        std::size_t a = std::strlen(data), b = std::strlen(rhs.data);
        char* joined = new char[a + b + 1];
        std::memcpy(joined, data, a);
        std::memcpy(joined + a, rhs.data, b + 1);
        Text result(joined);
        delete[] joined;
        return result;
    }
    bool operator==(const Text& rhs) const {
        return std::strcmp(data, rhs.data) == 0;
    }
    bool operator!=(const Text& rhs) const { return !(*this == rhs); }

    friend std::ostream& operator<<(std::ostream& out, const Text& t) {
        return out << t.data;
    }
};

int main() {
    Text a("Hello "), b("C++"), c(" World");
    Text result = a + b + c;
    std::cout << result << '\n';
    Text empty;
    std::cout << "Empty: [" << empty << "]\n";
```

```
    result = result; // self-assignment test
    std::cout << std::boolalpha << (Text("Hi") != Text("Bye")) << '\n';
}
```

Expected output:

```
Hello C++ World
Empty: []
true
```

2. Rules of operator overloading

- These operators cannot be overloaded: :: (scope resolution), . (member access), .* (pointer-to-member access), ?: (conditional operator) and sizeof.
- Overloading cannot change an operator's precedence, associativity or arity (number of operands).
- At least one operand must be a user-defined type; an overload cannot redefine an operator for two built-in types such as int + int.
- operator= must be a non-static member function. It controls assignment to the left-hand object.
- A friend overload is useful when a non-member operator needs private access, especially stream insertion where the left operand is an ostream.

3. Copies, concatenation and tests

For a + b + c, operator precedence and left associativity group the expression as (a + b) + c. The first concatenation produces a temporary of length n1+n2; the second combines that temporary with c. Depending on copy elision and the implementation, additional copy-constructor calls may be avoided. Instrument the copy constructor to count actual copies in the selected compiler build.

- Run with -fsanitize=address and test empty strings, long strings and self-assignment. The implementation allocates a fresh buffer before releasing the old one, avoiding self-assignment loss.
- Compare 100,000 concatenations with std::string using identical input sizes and optimization flags. Repeated concatenation can repeatedly copy earlier characters; reserve/append or a string builder is usually more efficient.
- Save actual sanitizer output and benchmark timings. Do not report invented values.

4. Non-overloadable operators: rationale

The listed operators are fundamental to parsing, member access, type layout or compile-time expression structure. Allowing user-defined replacement would undermine the language grammar or basic compile-time semantics.

5. Alternative concepts and complexity

Replacing Text with std::string removes the need to manually implement allocation, copying and destruction; std::string provides value semantics and follows the Rule of Zero. A custom Text may still be justified for a specialised representation, embedded environment, interoperability requirement or learning resource ownership.

For left-associated a+b+c, with lengths n1, n2 and n3, the first concatenation copies approximately n1+n2 characters and the second copies approximately (n1+n2)+n3 characters. Total character-copy work is O(n1+n2+n3) for this two-operation expression, with an intermediate temporary of length n1+n2

and final result of length n1+n2+n3. An implementation of operator+ using += on a by-value left operand can reuse that operand's buffer where capacity permits, but it still creates/returns a result object and may allocate. Exact temporary counts depend on copy elision and move semantics.

Q7 – Design a Complex-Number Class from First Principles (10 Marks)

Stage 1 – Interpret the problem

A complex impedance contains a real component and an imaginary component. Required data are real part, imaginary part, angular frequency ω, inductance L, capacitance C and resistance R. Required operations are construction, addition, subtraction, multiplication, division, equality, rectangular printing and magnitude.

Requirement	**Measurable acceptance condition**
Construction	Default, real-only and real-plus-imaginary constructors create the requested components.
Arithmetic	+,-,*,/ agree with hand calculations within a chosen floating-point tolerance.
Division safety	Division by a zero-magnitude denominator is detected and reported safely.
Circuit models	Series impedance is the sum; parallel impedance follows (Z1*Z2)/(Z1+Z2).
Display	Print real and imaginary components and magnitude.

Component formulas: resistor ZR = R + 0j; inductor ZL = 0 + jωL; capacitor ZC = 0 − j/(ωC), for ω > 0 and C > 0.

Stage 2 – Class design and formulas

- Store double real and imag. Constructors: Complex(), Complex(double real), and Complex(double real, double imag).
- Use member +, -, *, / and == for operations on the left-hand Complex object. Use friend << for output because the left operand is std::ostream.
- Addition: (a+jb)+(c+jd) = (a+c)+j(b+d). Subtraction: (a-c)+j(b-d).
- Multiplication: (ac-bd)+j(ad+bc). Division: [(ac+bd)+j(bc-ad)]/(c²+d²).
- Magnitude: sqrt(real²+imag²). Equality for floating-point values should use a tolerance rather than exact equality.

Stage 3 – Implementation and verification

```
#include <cmath>
#include <iomanip>
#include <iostream>
#include <stdexcept>

class Complex {
    double re, im;
public:
    Complex() : re(0.0), im(0.0) {}
    Complex(double real) : re(real), im(0.0) {}
    Complex(double real, double imag) : re(real), im(imag) {}

    double real() const { return re; }
```

```
    double imag() const { return im; }
    double magnitude() const { return std::hypot(re, im); }

    Complex operator+(const Complex& z) const {
        return Complex(re + z.re, im + z.im);
    }
    Complex operator-(const Complex& z) const {
        return Complex(re - z.re, im - z.im);
    }
    Complex operator*(const Complex& z) const {
        return Complex(re*z.re - im*z.im, re*z.im + im*z.re);
    }
    Complex operator/(const Complex& z) const {
        double denominator = z.re*z.re + z.im*z.im;
        if (denominator == 0.0)
            throw std::domain_error("Division by zero impedance");
        return Complex((re*z.re + im*z.im)/denominator,
                       (im*z.re - re*z.im)/denominator);
    }
    bool operator==(const Complex& z) const {
        const double eps = 1e-9;
        return std::abs(re-z.re) < eps && std::abs(im-z.im) < eps;
    }
    friend std::ostream& operator<<(std::ostream& out, const Complex& z) {
        out << std::fixed << std::setprecision(4)
            << z.re << (z.im < 0 ? " - j" : " + j") << std::abs(z.im);
        return out;
    }
};

int main() {
    const double omega = 100.0, R = 10.0, L = 0.1, C = 0.001;
    Complex zr(R, 0);
    Complex zl(0, omega * L);
    Complex zc(0, -1.0 / (omega * C));

    Complex series = zr + zl + zc;
    Complex z1 = zr + zl;
    Complex z2 = zc;
    Complex parallel = (z1 * z2) / (z1 + z2);

    std::cout << "ZR = " << zr << '\n';
    std::cout << "ZL = " << zl << '\n';
    std::cout << "ZC = " << zc << '\n';
    std::cout << "Series RLC = " << series
              << ", magnitude = " << series.magnitude() << '\n';
    std::cout << "Parallel (Z1||Z2) = " << parallel
              << ", magnitude = " << parallel.magnitude() << '\n';

    try {
        Complex zero;
        Complex invalid = zr / zero;
        std::cout << invalid << '\n';
    } catch (const std::domain_error& e) {
        std::cout << "Safe error: " << e.what() << '\n';
    }
}
```

Hand verification for the chosen example

Using $\omega = 100$ rad/s, $R = 10\ \Omega$, $L = 0.1$ H and $C = 0.001$ F: $Z_R = 10 + j0\ \Omega$; $Z_L = j(100 \times 0.1) = j10\ \Omega$; $Z_C = -j[1/(100 \times 0.001)] = -j10\ \Omega$. Series total $= 10 + j0\ \Omega$ and magnitude $= 10\ \Omega$.

For the parallel example, $Z_1 = Z_R + Z_L = 10 + j10\ \Omega$ and $Z_2 = Z_C = -j10\ \Omega$. $Z_1 + Z_2 = 10 + j0\ \Omega$; $Z_1 \times Z_2 = 100 - j100\ \Omega$; therefore $Z_{parallel} = 10 - j10\ \Omega$, with magnitude $\sqrt{200} \approx 14.1421\ \Omega$. Division by Complex(0,0) must print the safe error message.

Stage 4 – Defend the design

Aspect	**Custom Complex**	**std::complex<double>**
Operations	Must implement and test manually	Standard arithmetic already provided
Time per arithmetic operation	O(1)	O(1)
Storage	Two doubles	Two doubles in typical implementations
Division by zero	Explicit check and chosen error policy	Floating-point library semantics; caller should still validate denominator if application requires an error
Accuracy	Uses double floating-point arithmetic	Also uses double floating-point arithmetic

The constructors support zero impedance, a purely real impedance and a general complex impedance. Arithmetic operators express circuit equations naturally. One limitation is that the implementation uses double precision and a fixed absolute equality tolerance; applications with very large or very small impedances may need scale-aware tolerances and explicit validation of physical inputs such as zero frequency or zero capacitance.

Alternative: std::complex<double> and complexity

```
#include <complex>
using ComplexStd = std::complex<double>;
ComplexStd zr(R, 0.0);
ComplexStd zl(0.0, omega * L);
ComplexStd zc(0.0, -1.0 / (omega * C));
ComplexStd series = zr + zl + zc;
ComplexStd parallel = (z1 * z2) / (z1 + z2);
```

Both representations use O(1) time per arithmetic operation and O(1) storage (two floating-point components). std::complex<double> reduces implementation and testing work and provides standardised operations. The custom class is useful for learning, adding application-specific validation or enforcing a domain-specific error policy. Floating-point accuracy is broadly comparable when both use double; neither is exact decimal arithmetic.